

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Akhir
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengenalan Mikroprosesor

Abrar Rafi Dwianto - 5024241046

2026

1 Langkah-Langkah Percobaan

1. Percobaan 1 Pengenalan STM32 Blackpil (blink)
   pada percobaan pertama ini kami mencoba memrogram stm32 menggunakan platformio. STM32 kami program secara sederhana yaitu membuat lampu pada stm blink. untuk flashnya kami menggunakan ST-Link.

2. Percobaan 2 Pengenalan ESP32 dengan Framework ESP-IDF (Blink)
   pada percobaan ini pula kami mempelajari mengenai esp32 menggunakan framework esp-idf menggunakan platformio. disini kami memrogram agar lampu pada modul esp dapat berkedip. setelah kita program untuk mengeflashnya menggunakan kabel usb

3. Percobaan 3 Pull-up dan Pull-down: Eksternal vs Internal (ESP32 + Framework Arduino)
   pada percubaan ke 3 ini kami merangkai rangkaian pull up dan pull down pada button. kami kemudian memrogram esp32 menggunakan framework arduino untuk membaca input dari button tersebut. setelah dicatat hasilnya kami kemudian melepas semua resistor dan menghubungkan langsung button ke esp32 dan menulisprogram untuk menerapkan pull-up dan pull-down internal.

4. Percobaan 4 Debouncing pada Input Tombol GPIO (ESP32 + Framework Arduino)
   Pada percobaan ini kami menulis program pada esp32 dengan menggunakan framework arduino untuk membaca input dari button namun dengan menerapkan debouncing. mula-mula kita rangkai rangkaiannya kemudian menuliskan programnya lalu memflashnya.

5. Percobaan 5 Akses GPIO Analog: Pembacaan ADC dengan LDR (ESP32 + Framework Arduino)
   pada percobaan ini dirangkai rangkaian dimana sensor LDR disambungkan ke sensor ADC pada esp32. kemudian kami menulis program untuk membaca input ADC pada esp32 dan memflashnya.

2 Analisis Hasil Percobaan

1. Percobaan 1 Pengenalan STM32 Blackpil (blink)
   pada percobaan pertama ini kami mencoba memrogram stm32 menggunakan platformio untuk mengedipkan lampu yang ada pada esp32. untuk kodenya seperti berikut

```
1  #include <Arduino.h>
2
3  #define LED_PIN PC13
4
5  void setup()
6  {
7    pinMode(LED_PIN, OUTPUT);
8  }
9
10 void loop()
11 {
12   digitalWrite(LED_PIN, LOW);  // LED onboard aktif LOW -> LOW berarti menyala
13   delay(500);
```

```
14    digitalWrite(LED_PIN, HIGH); // HIGH berarti mati
15    delay(500);
16  }
17
```

bisa dilihat pada kode ada digital write low dan digital write high dengan delay 500ms atau 0.5 detik dimana ini akan membuat led akan berkedip hidup dan mati dengan jeda 0.5 detik.

2. Percobaan 2 Pengenalan ESP32 dengan Framework ESP-IDF (Blink)
   pada percobaan ini pula kami mempelajari mengenai esp32 menggunakan framework esp-idf menggunakan platformio. disini kami memrogram agar lampu pada modul esp dapat berkedip. Untuk programnya seperti berikut

```
1  #include "freertos/FreeRTOS.h"
2  #include "freertos/task.h"
3  #include "driver/gpio.h"
4
5  #define LED_PIN GPIO_NUM_2
6
7  void app_main(void)
8  {
9    gpio_reset_pin(LED_PIN);
10   gpio_set_direction(LED_PIN, GPIO_MODE_OUTPUT);
11
12   while (1)
13   {
14     gpio_set_level(LED_PIN, 1);
15     vTaskDelay(pdMS_TO_TICKS(500));
16     gpio_set_level(LED_PIN, 0);
17     vTaskDelay(pdMS_TO_TICKS(500));
18   }
19 }
20
```

dapat di lihat pada kode program esp32 menggunakan framework esp-idf, syntaxnya sangat berbeda dengan menggunakan stm32 dengan framework arduino. ini dikarenakan kedia framework ini dibuat dengan tujuan berbeda. framework arduino di desain lebih mudah untuk dipahami, berbeda dengan framework esp-idf yang lebih memberikan kebebasan dalam menulis program yang jauh lebih low level daripada framework arduino.

3. Percobaan 3 Pull-up dan Pull-down: Eksternal vs Internal (ESP32 + Framework Arduino)
   pada percobaan ini pertama kami merangkai rangkkaian pull-up dan pull-down secara external untuk membaca input button. Di mana rangkaian pull-up dan pull-down sendiri berfungsi untuk mencegah input mengambang. rangkaian pull-up dan pull-down external ini kemudian dibandingkan dengan pull-up dan pull-down internal bawaan dari esp32. Pada percobaan ini untuk pull-up dan pull-down esternal terkadang terdapat sedikit noise karena komponen kemungkinan ada yang kurang menancap namun kami tetap mendapatkan hasil yang sesuai yaitu ketiika tombol tidak ditekan outputnya juga tidak di tekan. Berbeda dengan menggunakan pull-up dan pull-down internal yang bahkan tidak ada noise sama sekali. pada percobaan ini untuk rangkaian pull-up apabila button tidak ditekan maka kondisi idlenya adalah high, berkebalikan dengan rangkaian pull-down.

4. Percobaan 4 Debouncing pada Input Tombol GPIO (ESP32 + Framework Arduino)
   Pada percobaan ini masih menggunakan button untuk membaca input namun dengan menerapkan debouncing. Debouncing sendiri berfungsi untuk menghilangkan kondisi osilasi antara 0 ke 1 ketika tombol ditekan akibat pengaruh mekanis dengan cara memberikan jeda pembacaan dalam kurun waktu tertentu. Pada percobaan ini kami membandingkan debounce delaynya dari 10ms, 50ms dan 200ms. Pada 10ms debounce sebenarnya berhasil dimana beberapa kondisi ketika menekan jika taanpa deounce terdeteksi tombol ditekan 2 kali sementara yang dengan edbounce terbacanya masih 1 kali. tapi masih delay terlalu cepat karena terkadang masih terbaca 2 kali. Untuk yang 50ms output debounce delaynya menurut saya sudah cukup. Untuk yang 200ms delaynya terlalu lama yang membuat respons dari buttonya semakin lama pula.

5. Percobaan 5 Akses GPIO Analog: Pembacaan ADC dengan LDR (ESP32 + Framework Arduino)
   Pada percobaan ini kami membaca input secara analog dari LDR di mana ketika dilakukan percobaan nilainya berubah-ubah sesuai intensitas cahayanya. Semakin terang cahayanya maka tegangannya semakin tinggi dan sebaliknya. Pada output serial monitor pembacaan nilai rawnya berkisar dari 0 hingga 4095. hal ini dikarenakan resolusi ADC esp32 adalah 12 bit sehingga jika kita bagi dengan 4095 dan dikalikan dengan maksimum voltase yang kita miliki maka outputnya adalah tegangan aslinya.

3 Hasil Tugas Modul

3.1 Matrix Pushbutton 3x3 Mengendalikan Matrix LED 3x3:

disini saya menggunakan wokwi untuk simulasinya. kodenya seperti berikut:

```
    #include <Arduino.h>

#define BUTTON_PIN_1 14
#define BUTTON_PIN_2 32
#define BUTTON_PIN_3 33
#define BUTTON_PIN_4 25
#define BUTTON_PIN_5 26
#define BUTTON_PIN_6 27

#define LED_PIN_1 19
#define LED_PIN_2 18
#define LED_PIN_3 5
#define LED_PIN_4 17
#define LED_PIN_5 16
#define LED_PIN_6 4

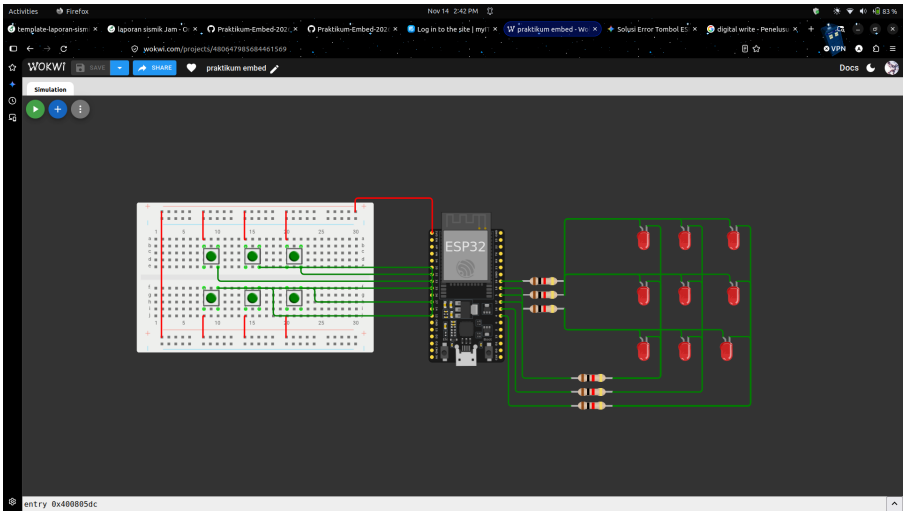
#define DEBOUNCE_DELAY 10 // ms

struct ButtonCondition{
    int lastFlickerableState = LOW;
    int stableState = LOW;
    unsigned long lastDebounceTime = 0;
    int switchStatus = 0;
};
```

```
25
26 ButtonCondition button1;
27 ButtonCondition button2;
28 ButtonCondition button3;
29 ButtonCondition button4;
30 ButtonCondition button5;
31 ButtonCondition button6;
32
33 void read_button(int pin, ButtonCondition* button){
34     int reading = digitalRead(pin);
35
36     if (reading != button->lastFlickerableState)
37     {
38         button->lastDebounceTime = millis();
39     }
40     if ((millis() - button->lastDebounceTime) > DEBOUNCE_DELAY)
41     {
42         if (reading != button->stableState)
43         {
44             button->stableState = reading;
45             // Serial.printf("pin: %d, %d\n", pin, button->stableState);
46             if (button->stableState == LOW){
47                 button->switchStatus = !button->switchStatus;
48             }
49         }
50     }
51     button->lastFlickerableState = reading;
52 }
53
54 void setup() {
55   Serial.begin(115200);
56   pinMode(BUTTON_PIN_1, INPUT_PULLDOWN);
57   pinMode(BUTTON_PIN_2, INPUT_PULLDOWN);
58   pinMode(BUTTON_PIN_3, INPUT_PULLDOWN);
59   pinMode(BUTTON_PIN_4, INPUT_PULLDOWN);
60   pinMode(BUTTON_PIN_5, INPUT_PULLDOWN);
61   pinMode(BUTTON_PIN_6, INPUT_PULLDOWN);
62
63   pinMode(LED_PIN_1, OUTPUT);
64   pinMode(LED_PIN_2, OUTPUT);
65   pinMode(LED_PIN_3, OUTPUT);
66   pinMode(LED_PIN_4, OUTPUT);
67   pinMode(LED_PIN_5, OUTPUT);
68   pinMode(LED_PIN_6, OUTPUT);
69 }
70
71 void loop() {
72     read_button(BUTTON_PIN_1, &button1);
73     read_button(BUTTON_PIN_2, &button2);
74     read_button(BUTTON_PIN_3, &button3);
75     read_button(BUTTON_PIN_4, &button4);
76     read_button(BUTTON_PIN_5, &button5);
77     read_button(BUTTON_PIN_6, &button6);
78
```

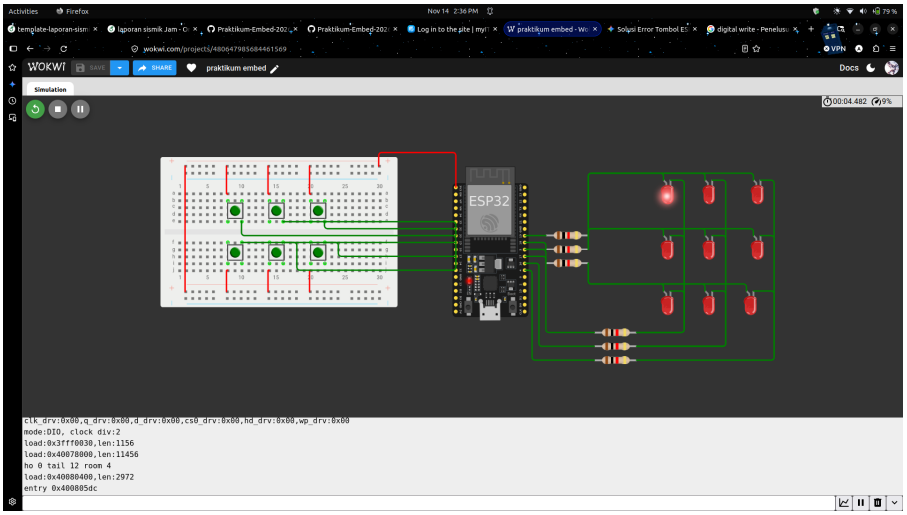
```
79      // Serial.printf("switch state button 1: %d\n", button1.switchStatus);
80      // Serial.printf("switch state button 2: %d\n", button2.switchStatus);
81      // Serial.printf("switch state button 3: %d\n", button3.switchStatus);
82      // Serial.printf("switch state button 4: %d\n", button4.switchStatus);
83      // Serial.printf("switch state button 5: %d\n", button5.switchStatus);
84      // Serial.printf("switch state button 6: %d\n", button6.switchStatus);
85
86      digitalWrite(LED_PIN_1, button1.switchStatus);
87      digitalWrite(LED_PIN_2, button2.switchStatus);
88      digitalWrite(LED_PIN_3, button3.switchStatus);
89      digitalWrite(LED_PIN_4, !button4.switchStatus);
90      digitalWrite(LED_PIN_5, !button5.switchStatus);
91      digitalWrite(LED_PIN_6, !button6.switchStatus);
92
93  }
```

rangkaainannya seperti berikut



Gambar 1: rangkaian tugas modul

ini apabila baris 1 dan kolom 1 dinyalakan:



Gambar 2: menyalakan led (1,1)

3.2 Pertanyaan Analisis:

1. karena tombol secara mekanis akan menimbulkan noise ketika ditekan dimana kontaknya itu tidak selalu bersentuhan, akibatnya bouncing high low terjadi yang bisa saja nanti membuat LED berkedip-kedip sehingga tetap membutuhkan debouncing.

2. dalam pembacaan, saya menggunakan konfigurasi pull-down dimana akan bernilai 0 jika tombol tidak ditekan. kondisi ini tidak boleh dibiarkan mengambanga agar pembacaan tetap sesuai dengan kondisi dari button aslinya dan tidak osilasi.

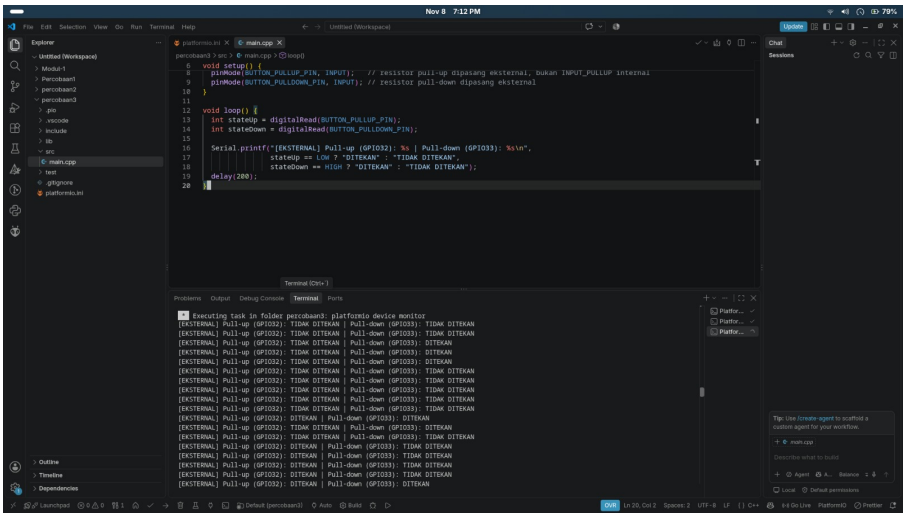
3. jika digantikan dengan sensor ldr, karena membaca menggunakan digital read ketika voltase ldr lebih dari threshold high pin stm maka pembacaan akan langsung bernilai 1 dan sebaliknya. inilah mengapa adc dapat merepresentasikan lebih dari 2 kondisi karena nilainya berupa analog atau floating point.

4 Kesimpulan

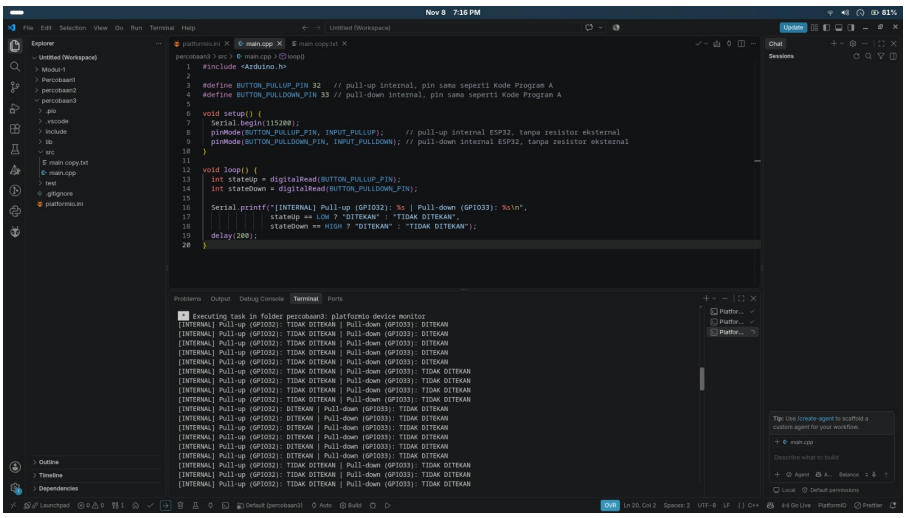
esp32 dan stm32 merupakan jenis mikrokontroler yang dapat digunakan untuk membuat sebuah sistem yang berinteraksi langsung dengan perangakat elektronik seperti membaca sensor atau menggerakan sesuatu. keduanya dapat di program menggunakan platformio dengan framwork arduino maupun framwork espidf. dalam membaca sensor penting untuk mengkombinasikan rangkaian pull-up dan pull-down untuk mencegah floating. dalam kasus tombol perlu juga ditambahkan debouncing untuk meminimalisir error mekanik dari button yang membuat button telihat seperti ditekan berulang padahal sebenarnya hanya ditekan sekali. selain itu esp dapat membaca input berupa sinyal digital maupun analog menggunakan ADC pada beberapa pinnya.

5 Lampiran

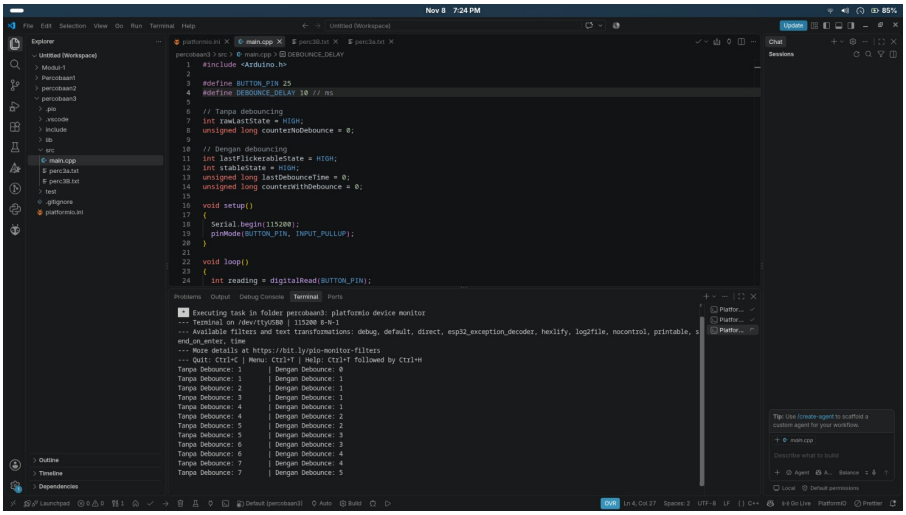
5.1 output serial monitor



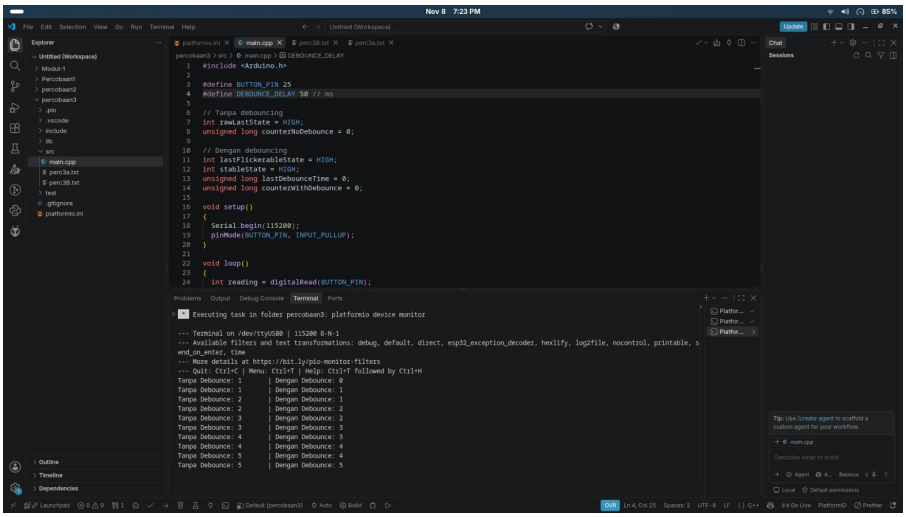
Gambar 3: Serial monitor percobaan 3 pull-up dan pull-down external



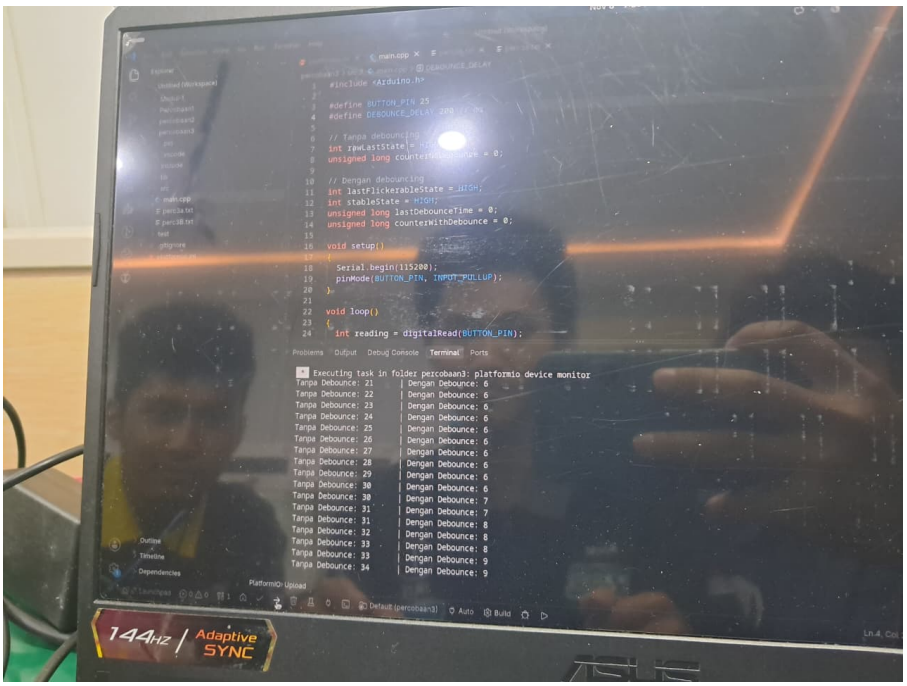
Gambar 4: Serial monitor percobaan 3 pull-up dan pull-down external



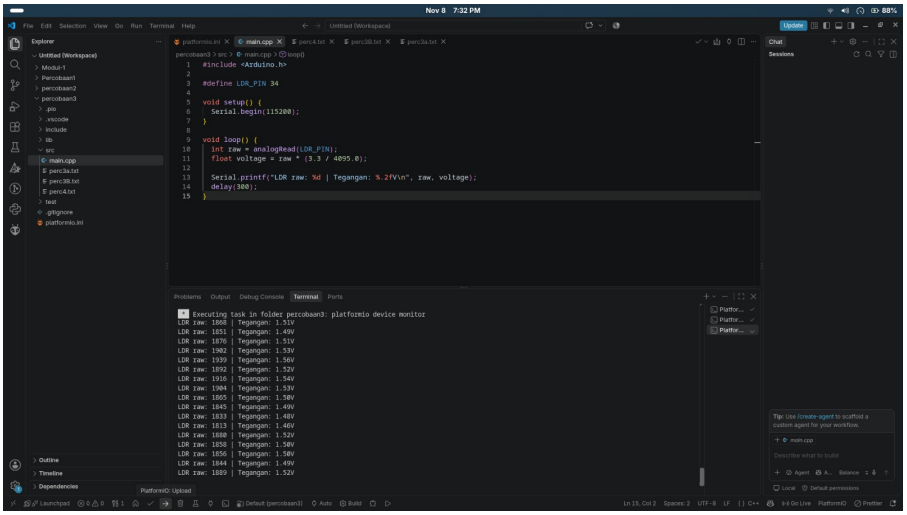
Gambar 5: Serial monitor percobaan 4 debounce 10ms



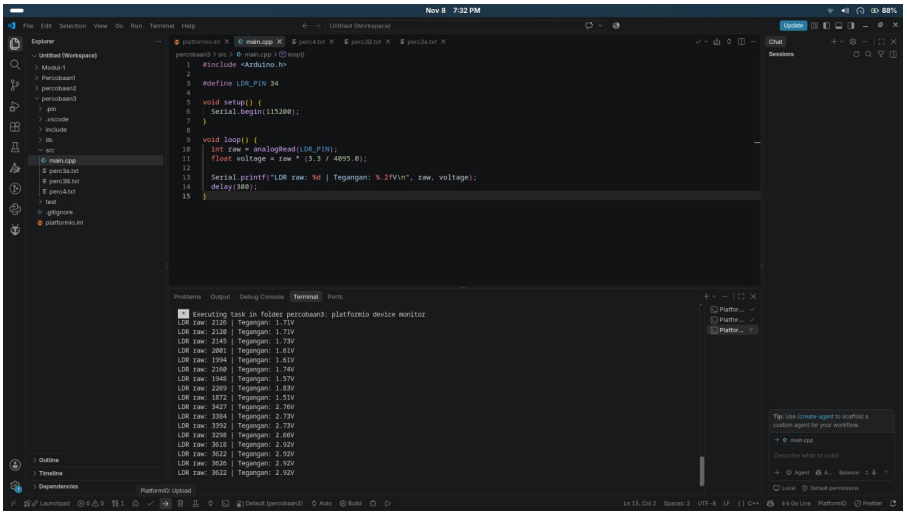
Gambar 6: Serial monitor percobaan 4 debounce 50ms



Gambar 7: Serial monitor percobaan 4 debounce 200ms

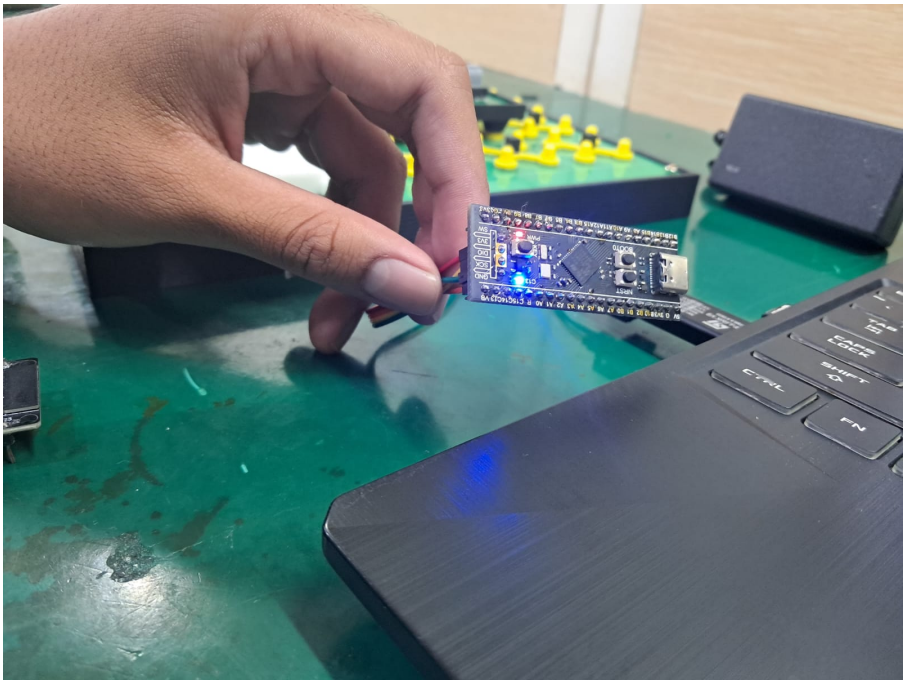


Gambar 8: Serial monitor percobaan 5 LDR ditutup

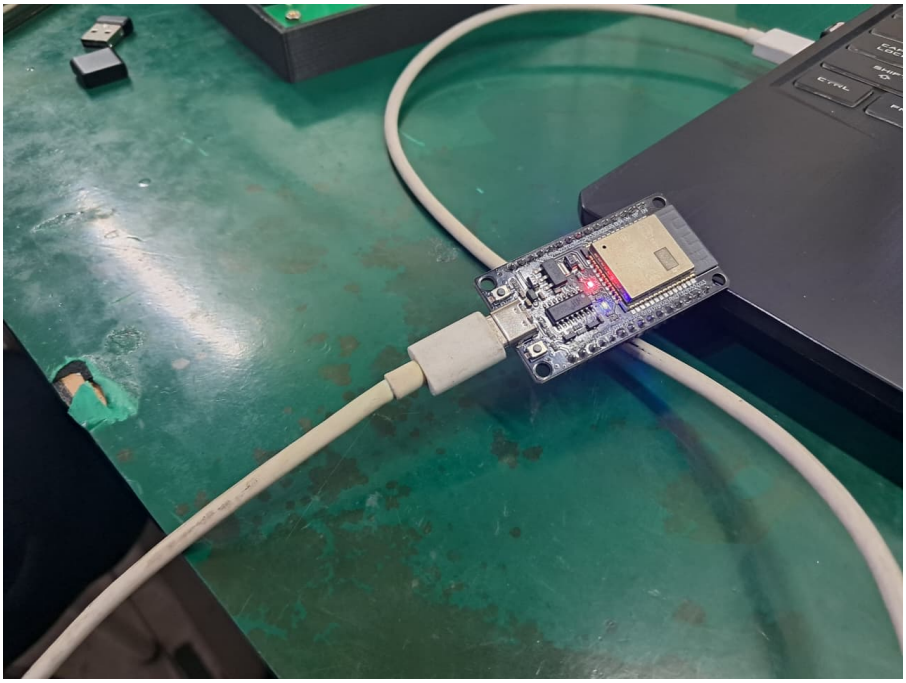


Gambar 9: Serial monitor percobaan 5 LDR dibuka

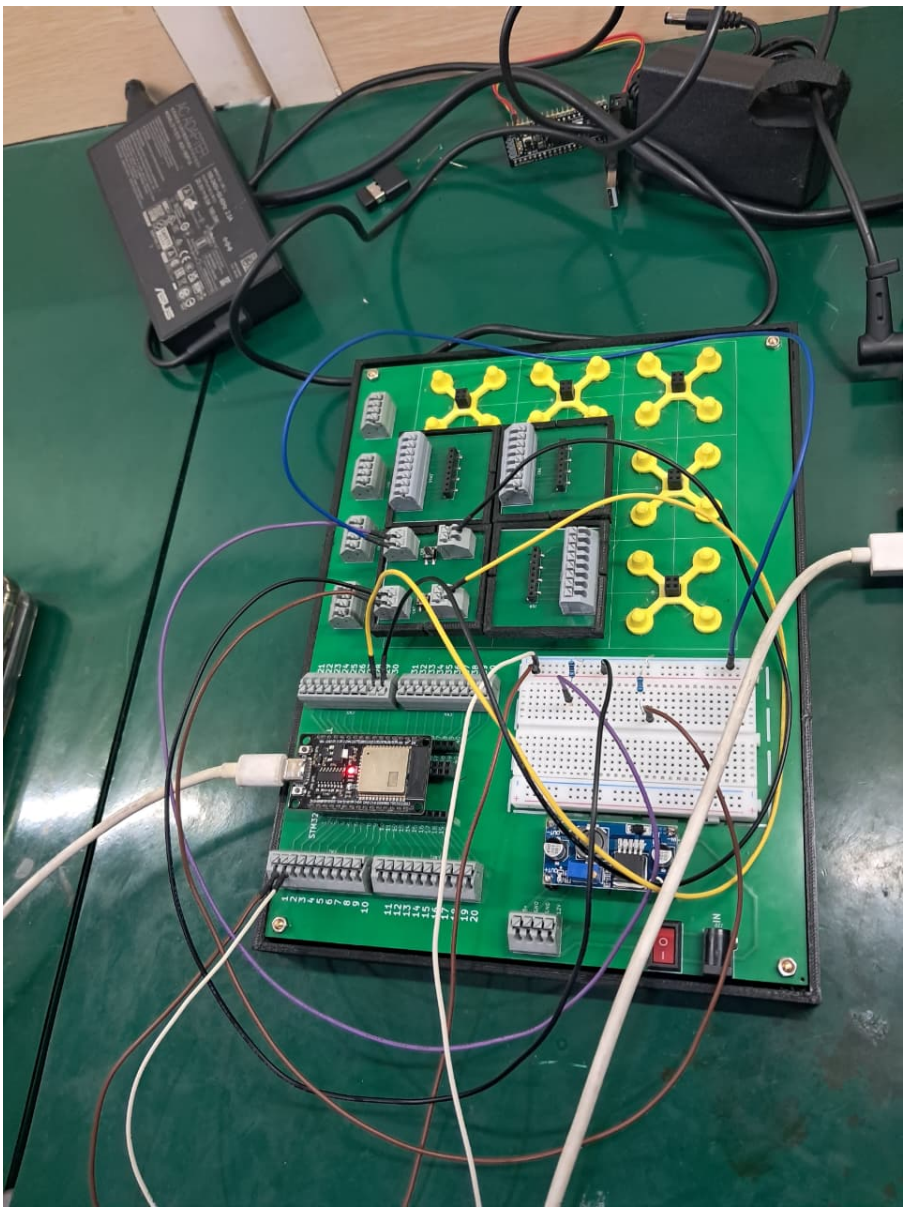
5.2 Dokumentasi saat praktikum



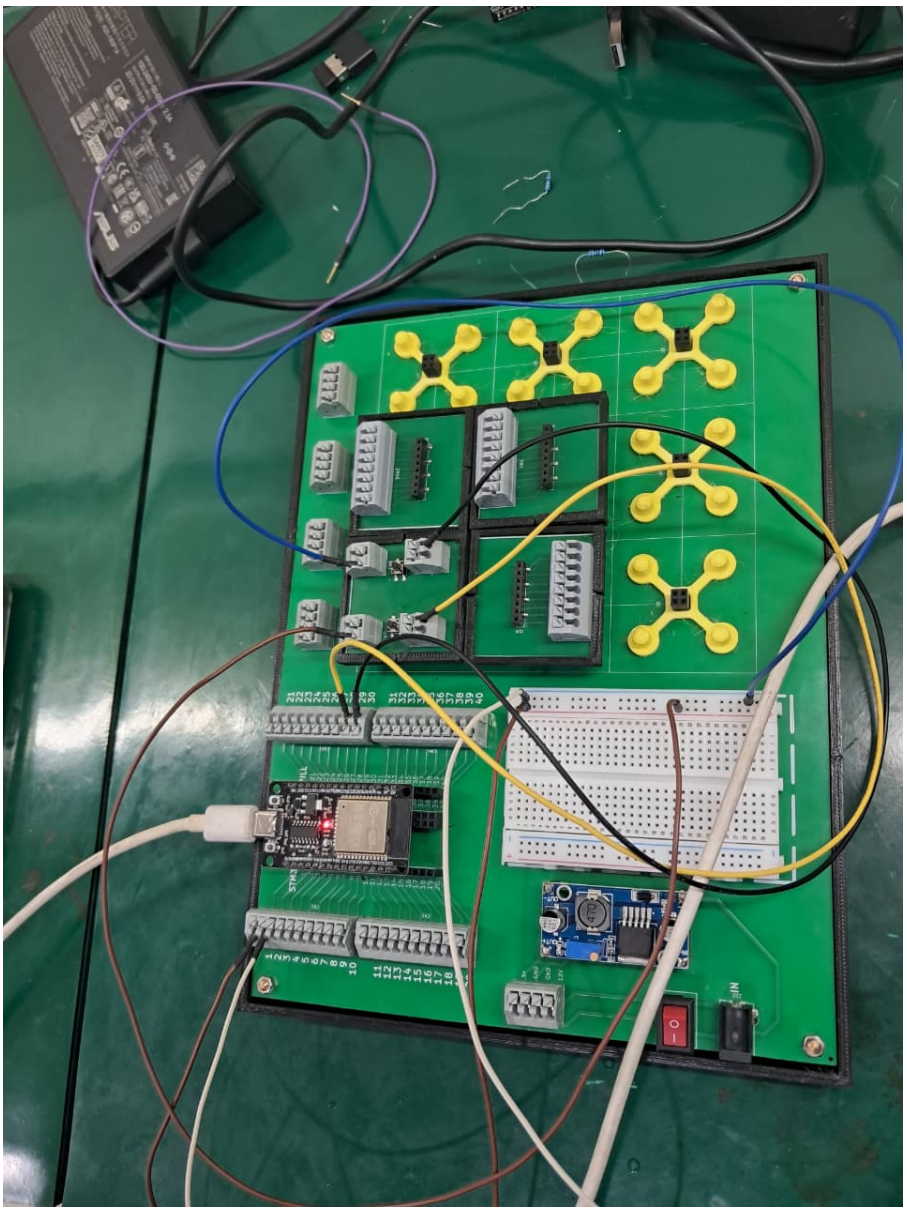
Gambar 10: bukti percobaan 1 blink stm32



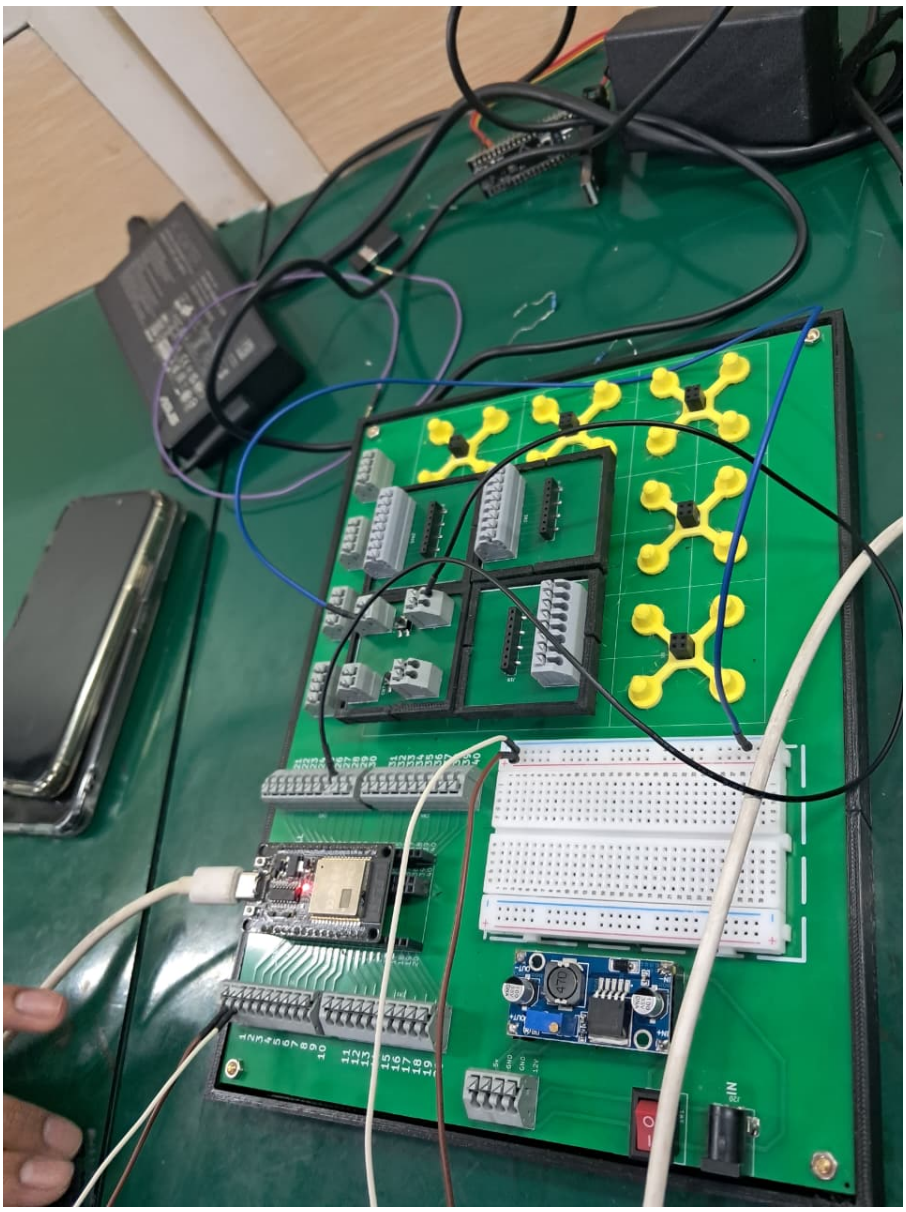
Gambar 11: bukti percobaan 2 blink esp32



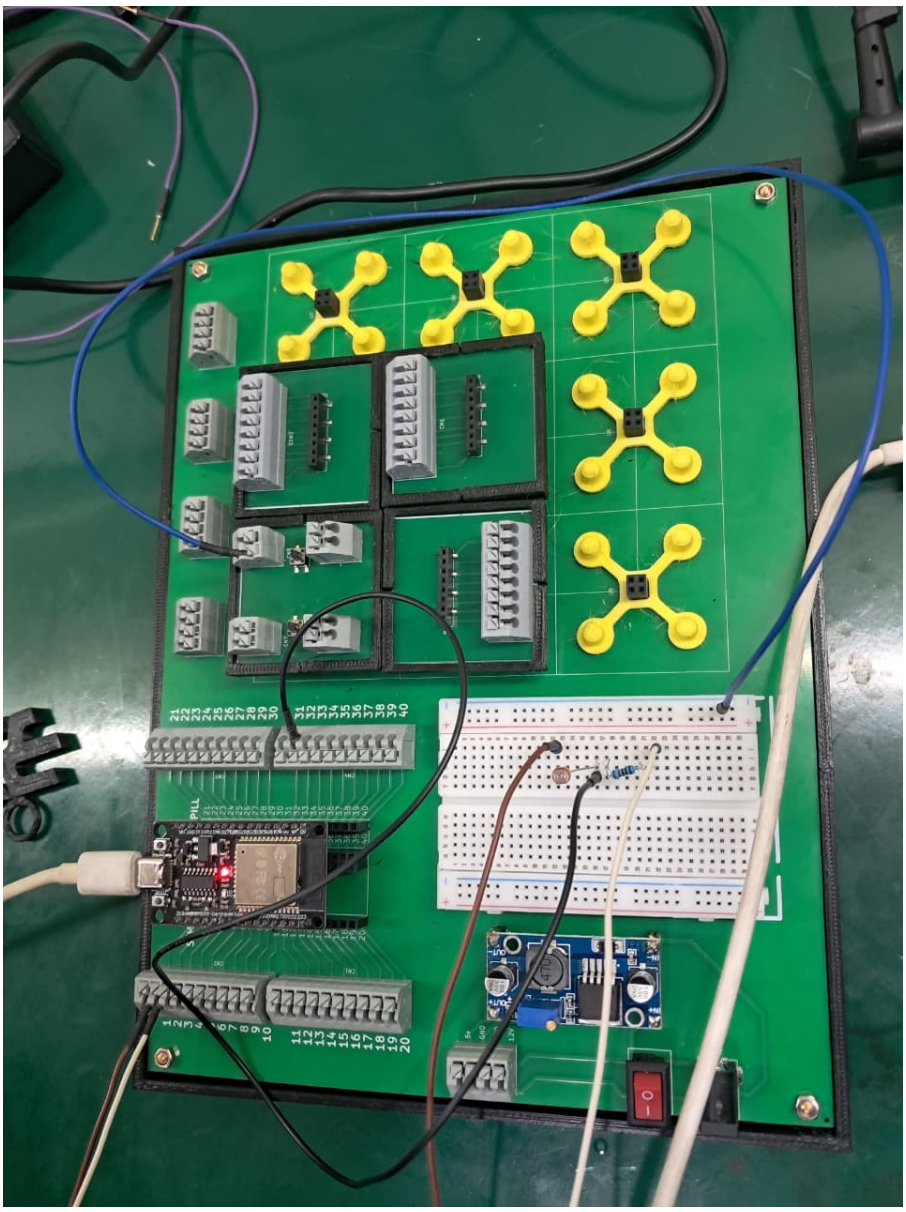
Gambar 12: bukti percobaan 3 external pull-up pull-down



Gambar 13: bukti percobaan 3 external pull-up pull-down



Gambar 14: bukti percobaan 4 debounce



Gambar 15: bukti percobaan 5 LDR



Gambar 16: foto bersama